



발표 대본

표지 · 약 20초

안녕하십니까. 과제 3, 표적 궤적 모의 발표를 시작하겠습니다.

내용은 세 덩어리입니다. 표적을 어떤 값으로 정의했는지, 그 표적을 시간에 따라 어떻게 움직였는지, 그리고 그 결과를 어떻게 확인했는지입니다.

INDEX

표적 궤적 모의
TargetSim

Janghwan Kim (Harley)

01

표적을 정의한다

표적 · 기동을 어떤 값으로
적을 것인가

5 ~ 8

02

시간에 따라 움직인다

한 스텝에 쓰는 공식과
그 공식이 필요한 이유

9 ~ 13

03

결과를 확인한다

60 초 궤적 · 기동 시연
운용 화면

14 ~ 17

2

발표 대본

목차 · 약 30초

1장에서 표적과 기동을 어떤 값으로 적을지를 정합니다. 여기서 정한 값이 그대로 구조체가 됩니다.

2장이 구현의 중심입니다. 한 스텝, 그러니까 0.1초를 전진시키는 데 필요한 공식 두 개와, 그 공식이 왜 필요한지를 코드와 같이 보겠습니다.

3장에서 실제로 돌린 결과를 보여 드리겠습니다.

좌표계와 좌표변환 자체는 Part 2 에서 이미 다뤘기 때문에, 이번에는 그 이론을 다시 설명하지 않고 필요한 자리에서 "여기서 이 변환을 쓴다" 정도로만 언급하겠습니다.

들어가며

무엇을 왜 만들었는가

표적이 실제로 어떻게 움직였는지 — 참값 — 를 만들어 내는 일입니다. 참값이 있어야 추적 결과가 맞는지 판정할 수 있습니다.

이 모의가 어디에 쓰이는가



구현은 세 단계로 나뉘었다



3

발표 대본

도입 · 약 70초

먼저 이 모의가 어디에 쓰이는 물건인지부터 말씀드리겠습니다.

맨 위 흐름을 봐 주십시오. 표적 궤적 모의가 있고, 그 뒤에 측정 모의, 추적 필터, 전시와 통제가 붙습니다. 이번 과제는 그 중 맨 앞 블록 하나입니다.

이 블록이 하는 일은 표적이 실제로 어떻게 움직였는지, 즉 참값을 만들어서 뒤쪽에 넘겨주는 것입니다. 뒤에 있는 추적 필터는 잡음 섞인 측정값만 보고 표적의 위치를 추정하는데, 그 추정이 맞았는지 틀렸는지는 참값을 알아야 판정할 수 있습니다. 실제 바다에서 시험을 하면 배와 비행기를 띄워야 하고, 같은 상황을 두 번 만들 수 없고, 표적이 정확히 어디 있었는지 알려면 기준 장비가 또 필요합니다. 모의는 제가 넣은 값이 곧 정답이라 그런 문제가 없습니다.

가운데가 구현 단계입니다. 표적을 정의하고, 시간에 따라 움직이고, 결과를 확인한다. 이 세 단계가 그대로 오늘 발표 순서입니다.

아래 띄는 과제 문서의 요구사항을 이 세 단계에 붙여 놓은 것입니다. 요구사항을 따로 한 장 잡아서 읽는 대신, 어느 단계에서 무엇을 만족시켰는지를 여기에 같이 표시했습니다.

들어가며

만든 것 — 계산과 화면을 나눈다

연산 엔진만 따로 빌드해 검증할 수 있게 했습니다. 오늘 보여 드리는 숫자는 모두 그 엔진을 돌려 얻은 것입니다.



4

발표 대본

도입 · 약 50초

만든 물건은 두 계층입니다.

아래 연산 계층이 순수 C로 된 TargetSimCore입니다. 이 중에서 CoordinateTransform과 matrixCalcLib는 제공받은 코드 원본이고, 제가 새로 쓴 것은 맨 위 TargetSim.c 하나입니다.

위 표현 계층이 MFC로 만든 화면입니다.

둘 사이에 오가는 것은 오른쪽 구조체 두 개뿐입니다. 설정을 담은 ST_SimConfig를 넣으면 시각마다의 상태인 ST_SimSample이 나옵니다. 공개 함수는 여섯 개고, UI는 Core안을 전혀 모릅니다.

이렇게 나눈 덕을 실제로 봤습니다. 오늘 보여 드리는 숫자를 뽑을 때 리눅스에서 gcc로 Core만 따로 컴파일해서 돌렸는데, MFC 쪽은 손댈 필요가 없었습니다.

01. 표적을 정의한다

표적을 어떤 값으로 적을 것인가

명세가 정한 7 개를 받아, 계산하기 좋은 9 개로 바꿔 들고 있습니다.

받는 값 — 명세가 정한 7 개

위도	32.12 °
경도	126.00 °
고도	300 m
속력	200 m/s
Roll	0 °
Pitch	0 °
Yaw	180 °

어디에 · 어디를 보고 · 얼마나 빠른가.
속도는 크기 하나만 받는다 — 방향은
자세각이 이미 들고 있기 때문이다.

내부에서 굴리는 값 — 상태 9 개

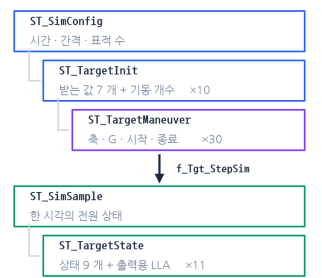
$\mathbf{p}^e$ ECEF 위치
 x, y, z [m]
 $\mathbf{v}^e$ ECEF 속도
 v_x, v_y, v_z [m/s]
 Θ 자세각
Roll, Pitch, Yaw [rad]

왜 ECEF 로 바꿔 두는가

· 미터 단위 직교좌표라 위치 전진이
덧셈 한 번으로 끝난다
· 명세가 지정한 갠신 좌표계다

출력할 때만 LLA 로 되돌린다.
좌표변환 함수는 Part 2 의 것을 그대로 쓴다.

구조체로 옮기면



입력과 출력을 완전히 같았다. 설정은 처음
한 번만 읽고, 스텝마다 바뀌는 것은 상태뿐이다.
10 × 30 은 고정 배열 — 실행 중 메모리 할당 실패가 없다.

5

발표 대본

1장 · 약 70초

표적 하나를 코드로 옮기려면 먼저 "무엇을 받고, 무엇을 들고 있을 것인가" 를 정해야 합니다.

왼쪽이 받는 값입니다. 명세가 정해 준 대로 일곱 개입니다. 어디에 있는가, 어디를 보고 있는가, 얼마나 빠른가. 여기서 눈여겨보실 부분은 속도를 크기 하나로만 받는다는 점입니다. 방향은 이미 자세각이 들고 있기 때문에 또 받을 필요가 없습니다. 오른쪽 위로 도는 배가 있으면, 그 배의 속도 벡터는 "속력 30" 과 "Yaw 270도" 두 개로 완전히 정해집니다.

가운데가 계산하는 동안 들고 있는 값입니다. 위치와 속도를 ECEF 로, 자세각 세 개를 합쳐서 아홉 개입니다.

ECEF 로 바꿔 두는 이유는 두 가지입니다. 하나는 미터 단위 직교좌표라서 위치를 앞으로 보내는 일이 덧셈 한 번으로 끝난다는 것. 위도와 경도로 바로 계산하면 "북쪽으로 1도" 가 위치마다 다른 거리라서 매번 다른 계산을 해야 합니다. 다른 하나는 명세가 ECEF 에서 갠신하라고 지정했다는 것입니다. 사람이 읽어야 하는 출력 시점에만 위경도로 되돌립니다.

오른쪽이 그걸 구조체로 옮긴 모습입니다. 받는 값과 나오는 값을 완전히 같았습니다. 설정은 처음 한 번만 읽고, 스텝마다 바뀌는 것은 상태쪽뿐입니다. 배열 크기를 10과 30으로 고정한 것은 명세가 준 상한이기도 하고, 실행 중에 메모리를 잡다가 실패하는 일을 아예 없애려는 것이기도 합니다.

01. 표적을 정의한다

기동을 어떤 값으로 적을 것인가

"12 초부터 16 초까지 오른쪽으로 8 G" 라는 지시가 구조체 한 줄이 됩니다.

기동 1 건 = 값 네 개

enTurnType Roll / Pitch / Yaw	어느 축으로 도는가
gravityValue 크기 = 세기, 부호 = 방향 [G]	얼마나 세게 도는가
startTime 이 시각 포함 [s]	언제부터
endTime 이 시각 직전까지 [s]	언제까지

기동이 없는 시간에는 각속도 0 — 직전한다.
구간이 서로 겹치면 계산 전에 걸러낸다.

기동 지시는 원래 "위도 32.1 에서 좌로 90 도" 처럼 좌표를 온다.
구조체에는 시각 칸만 있으므로, Core 를 한 번 돌려 그 좌표에
달는 시각을 찾아 start / end 로 옮겨 적었다.

왜 G 하나로 충분인가

필요한 것은 "1 초에 몇 도 도는가" 인데, 받는 값은 G 하나다.

$$a_c = \frac{v^2}{R} = V\omega = ng$$

원을 그리며 돌 때 필요한 가속도를 중력의 n 배로 적는다

$$\omega = \frac{ng}{V} \rightarrow \text{이 한 줄로 각속도가 정해진다}$$

$\omega = \text{gravityValue} * G_FORCE / \text{headingSpeed};$

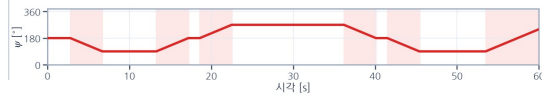
같은 4 G 라도

대함 30 m/s 74.9 °/s R 23 m 대공 200 m/s 11.2 °/s R 1020 m

분모에 속력이 있어 빠른 표적일수록 덜 꺾인다.

기동표 6 줄이 실제로 그려낸 방위각

색칠한 구간에서만 각이 돈다



6

발표 대본

1장 · 약 75초

다음은 기동입니다. "12초부터 16초까지 오른쪽으로 8G" 같은 지시를 코드가 다룰 수 있는 값으로 바꿔야 합니다.

왼쪽처럼 네 칸으로 정리했습니다. 어느 축으로 도는가, 얼마나 세게 도는가, 언제부터, 언제까지. 세기는 G 값의 크기로, 방향은 G 값의 부호로 받습니다. 구간은 시작은 포함하고 끝은 포함하지 않게 잡아서, 앞 기동의 끝과 다음 기동의 시작이 같은 숫자여도 겹침으로 보지 않습니다. 실제로 겹치는 표가 들어오면 계산을 시작하기 전에 거부합니다.

오른쪽이 오늘 처음 나오는 공식입니다. 표적을 실제로 돌리려면 "1초에 몇 도 도는가", 각속도가 필요한데, 우리가 받는 값은 G 하나입니다. 이 둘을 잇는 게 원운동 관계식입니다.

원을 그리며 도는 물체에 필요한 가속도는 속력의 제곱 나누기 반경이고, 이건 속력 곱하기 각속도와 같습니다. 그리고 그 가속도를 중력가속도의 n 배, 즉 n G 로 적는 겁니다. 세 개를 나란히 놓고 정리하면 각속도는 n G 를 속력으로 나눈 값이 됩니다. 아래 코드 한 줄이 정확히 그것입니다.

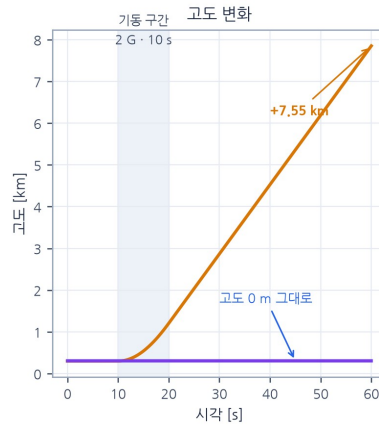
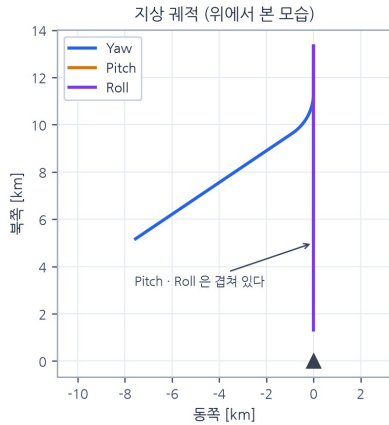
여기서 재미있는 게 분모에 속력이 있다는 점입니다. 같은 4G 라도 30 m/s 로 가는 배는 초당 75도씩 꺾어 도는데, 200 m/s 로 가는 항공기는 초당 11도밖에 못 돕니다. 선회 반경으로 보면 23미터 대 1020미터입니다. 빠를수록 크게 돈다는 게 식에 그대로 들어 있습니다.

아래 그래프가 실제로 돌려 본 결과입니다. 기동표 여섯 줄을 넣었더니 색칠한 구간에서만 방위각이 기울어졌고, 나머지 구간에서는 평평합니다. 기울어진 정도가 방금 그 각속도입니다.

01. 표적을 정의한다

축을 바꾸면 궤적이 어떻게 달라지는가

같은 표적에 같은 2G를 10초씩, 축만 바꿔 걸어 본 실행 결과입니다.



같은 표적 · 같은 2G · 같은 10s, 축만 바꿨다

Yaw
 ψ 가 118° 돌고 궤적이 휜다.
고도는 300 m 그대로.

Pitch
 θ 가 들려 7.55 km 상승.
지상 궤적은 거의 제자리.

Roll
 ϕ 만 변하고 궤적은 불변.
회전축이 곧 속도 방향이라 그렇다.

Roll이 궤적을 못 바꾸는 것은 구현 오류가 아니라 물리적으로 옳은 결과다.

7

발표 대본

1장 · 약 55초

회전축을 세 개 둔 이유를 한 번 확인해 봤습니다. 같은 표적에 같은 2G를, 같은 10초 동안, 축만 바꿔서 걸어 본 것입니다.

Yaw를 걸면 왼쪽 그림처럼 궤적이 옆으로 휩니다. 10초 동안 118도 돌았고, 고도는 300미터 그대로입니다.

Pitch를 걸면 지상에서 보면 거의 제자리인데, 오른쪽 고도 그래프가 7.5킬로미터 넘게 올라갑니다. 기수가 들려서 위로 날아간 겁니다.

Roll은 각도만 변하고 궤적이 전혀 안 바뀝니다. 처음에는 버그인 줄 알았는데, 생각해 보니 맞는 결과였습니다. Roll은 표적이 가는 방향을 축으로 삼아 도는 회전이라, 속도 벡터 자체는 그 축 위에 있어서 회전을 해도 제자리입니다. 팽이가 돌아도 팽이 축은 안 움직이는 것과 같습니다.

구현이 맞는지 보려고 돌린 건데, 물리적으로도 맞는 그림이 나왔습니다.

01. 표적을 정의한다

코드로 보는 구조체

입력 — TargetSim.h

```
// 기동 1건 . gravityValue 의 부호가 회전 방향이다 .
typedef struct
{
    EN_TurnType    enTurnType;    // 0~3
    FLOAT64        gravityValue;  // [G]
    FLOAT64        startTime;     // [s]
    FLOAT64        endTime;      // [s]
} ST_TargetManeuver;

// 위치는 위 / 경 / 고도 , 속도는 동체 x 축 방향 크기 .
typedef struct
{
    STRUCT_Coord_Lla    st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64             headingSpeed; // V_heading
    INT32               nManeuverNum;
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

출력 — TargetSim.h

```
// 한 스텝 계산은 ECEF 에서 , 출력은 LLA 로 .
// 둘을 함께 들고 있어 스텝마다 한 번만 변환한다 .
typedef struct
{
    FLOAT64        simTime;    // [s]
    STRUCT_Coord_Rect st_PosEcef; // [m]
    STRUCT_Coord_Rect st_VelEcef; // [m/s]
    STRUCT_Coord_Lla st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 . 표적 상태 — 이게 한 장의 표본이다 .
typedef struct
{
    FLOAT64        simTime;
    INT32          nStepIndex;
    ST_TargetState st_Platform;
    ST_TargetState st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

구간은 [시작, 종료)

끝과 다음 시작이 같아도 겹침이 아니다.
실제로 겹치면 계산 전에 거부한다.

계산용과 출력용을 함께

계산은 ECEF, 사람이 읽는 값은 LLA.
스텝마다 한 번만 바꿔 두 곳에서 쓴다.

플랫폼도 같은 구조체

기동이 없는 표적일 뿐이라, 각속도만 0 으로
넣어 같은 경로로 전진시킨다.

8

발표 대본

1장 · 약 60초

앞에서 정한 값들을 실제 헤더로 보시겠습니다.

왼쪽 위가 기동 하나입니다. 아까 말씀드린 네 칸이 그대로 있습니다.

왼쪽 아래가 표적 하나의 입력입니다. 초기 위치, 초기 자세, 속력 하나, 그리고 기동 배열입니다.

오른쪽 위가 계산하면서 들고 있는 상태입니다. ECEF 위치와 속도가 계산용이고, 그 아래 LLA 와 자세가 출력용 겸 다음 스텝 입력용입니다. ECEF 에서 위경도로 바꾸는 계산이 한 스텝에 한 번 들어가는데, 그 결과를 여기 같이 담아 두면 출력할 때 다시 바꿀 필요가 없습니다.

오른쪽 아래가 한 시각의 표본입니다. 플랫폼 하나와 표적 배열이 같이 들어갑니다.

아래 카드 세 개 중 마지막을 하나만 짚겠습니다. 플랫폼도 표적과 똑같은 구조체를 씁니다. 플랫폼은 기동이 없는 표적일 뿐이라, 각속도만 0으로 넣고 같은 함수로 전진시킵니다. 코드가 한 벌로 줄었습니다.

02. 시간에 따라 움직인다

한 스텝에 하는 일

이 다섯 단계가 `f_Tgt_StepSim` 한 번입니다. 601 번 반복하면 60 초 궤적이 됩니다.

0.1 초를 전진시키는 다섯 단계

이 다섯 줄이 `f_Tgt_StepSim` 한 번입니다. 601 번 반복하면 60 초 궤적이 됩니다.



9

발표 대본

2장 · 약 90초

여기가 오늘 발표의 중심입니다. 0.1초를 전진시키는 데 무엇을 하는지, 다섯 단계로 나눠 봤습니다. 이 다섯 줄이 `f_Tgt_StepSim` 함수 한 번이고, 601번 반복하면 60초 궤적이 됩니다.

1단계, 지금 시각이 어느 기동 구간에 들어 있는지 찾습니다. 들어 있으면 아까 그 공식으로 각속도를 계산하고, 안 들어 있으면 각속도가 0이라 직진합니다.

2단계, 그 각속도로 자세를 돌립니다. 각속도 곱하기 시간만큼 각도를 더하는 겁니다. 이것 먼저 하는 이유는, 방향이 정해져야 속도를 만들 수 있기 때문입니다.

3단계, 속도를 만듭니다. "기수 방향으로 속력 V" 라는 정보를 실제 ECEF 벡터로 바꾸는 단계입니다. 여기에 회전 행렬 두 개가 들어가는데, 다음 장에서 따로 보겠습니다.

4단계, 그 속도로 위치를 옮깁니다. 위치에 속도 곱하기 시간을 더하면 끝입니다. ECEF 가 미터 단위 직교좌표라서 이렇게 간단합니다. 이게 아까 ECEF 로 바뀌 든 이유입니다.

5단계, 결과를 위경도와 고도로 되돌려서 표본에 담습니다. 사람이 읽고, 뒤쪽 블록이 받아 쓰는 형식입니다.

아래 왼쪽에 한 줄 덧붙이면, 3단계와 4단계는 실제로 두 번 돌립니다. 반 스텝만 살짝 가 보고 거기서 속도를 다시 켜 다음, 그 속도로 한 스텝을 옮깁니다. 이유는 뒤에서 그림으로 보여 드리겠습니다.

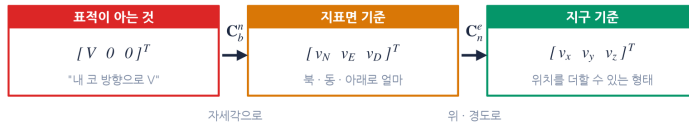
아래 오른쪽. 이 다섯 단계를 표적 수만큼 반복합니다. 표적끼리 서로 영향을 주지 않기 때문에 for 문 하나로 10대까지 그대로 늘어납니다.

02. 시간에 따라 움직인다

공식 ① — 속도 벡터를 만든다

과제가 따로 묻은 "동체 속도 V_{heading} 을 ECEF 의 V_x, V_y, V_z 로 바꾸는 방법" 에 대한 답입니다.

속도 벡터 만들기 — 받은 것은 숫자 하나, 필요한 것은 방향까지



$$\mathbf{v}^e = \mathbf{C}_n^e(\phi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

왜 이 식이 필요한가

- 입력은 속력 하나뿐이다 — 방향은 자세각이 들고 있다.
- 위치를 더하려면 위치와 같은 좌표계의 속도여야 한다.

두 회전행렬은 Part 2 에서 만든 함수를 그대로 가져다 썼다.
새로 만든 것은 "두 번 이어 붙인다" 는 이 한 줄뿐이다.

이 식이 맞게 돌아갔다는 근거

$$\|\mathbf{v}^e\| = V$$

회전은 방향만 바꾼다 — 601 표본 전부
입력 속력과 같게 나왔다.

함수 한 줄에서 주의한 곳

```
f_Trans_Ned_To_Ecef(
  vN, vE, vD,
  0.0, 0.0, 0.0, lat, lon)
```

이 함수는 기준점까지 더해 준다. 속도는 방향만
옮기면 되므로 기준점을 0 으로 넣는다. 위치를
옮길 때와 호출 방법이 다른 유일한 지점이다.

10

발표 대본

2장 · 약 70초

3단계에서 쓰는 공식입니다. 과제 문서에 괄호로 "동체 속도 값을 ECEF 의 V_x, V_y, V_z 로 바꾸는 방법을 고민해 보라, 안 해도 된다" 고 적혀 있던 항목인데, 이건 안 풀면 구현 자체가 안 돼서 정면으로 다뤘습니다.

문제는 이렇습니다. 우리가 받은 건 숫자 하나, 속력뿐입니다. 그런데 위치를 옮기려면 x, y, z 세 방향으로 각각 초당 몇 미터인지를 알아야 합니다. 방향 정보를 어디선가 만들어 붙여야 합니다.

위쪽 세 상자가 그 과정입니다. 표적 입장에서 "내 코 방향으로 V" 입니다. 이것 자세각으로 한 번 돌리면 지표면 기준으로 북쪽 얼마, 동쪽 얼마, 아래로 얼마가 나옵니다. 거기서 위도와 경도로 한 번 더 돌리면 지구 중심 기준 벡터가 됩니다. 회전 두 번이면 끝입니다.

아래 식이 그 두 번을 한 줄로 쓴 것입니다. 회전행렬 자체는 Part 2 에서 만든 함수를 그대로 가져다 썼기 때문에, 이번에 새로 작성한 건 "두 번 이어 붙인다" 는 이 한 줄입니다.

오른쪽 위가 이게 제대로 돌아갔다는 근거입니다. 회전은 방향만 바꾸고 크기는 건드리지 않습니다. 그러니까 변환을 아무리 거쳐도 속도 벡터의 크기는 처음에 넣은 속력과 같아야 합니다. 601개 표본을 전부 확인했는데 전부 입력값과 같았습니다. 이게 어긋나면 어딘가에서 행렬을 잘못 곱한 겁니다.

02. 시간에 따라 움직인다

코드 ① — 속도 벡터를 만든다

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번, 위경도로 한 번 돌려 ECEF 속도로 만든다.
// 회전만 거치므로 속력은 그대로 보존된다.
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다.
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
    st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다. 방향만 옮길 때는 기준점을 원점으로 준다.
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
    0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

기준점을 0 으로 주는 이유

이 함수는 회전에 더해 기준점만큼 평행이동까지 한다. 속도는 "위치" 가 아니라 "방향" 이라 그 이동분을 빼야 한다.

인자 순서를 주석에 남긴 이유

제공받은 함수가 Roll, Yaw, Pitch 순서로 받는다. 자세각 구조체의 필드 순서와 달라 한 번 틀렸던 자리다.

제대로 돌아갔는지 확인

회전은 방향만 바꾸고 크기는 건드리지 않는다. 601 표본 전부터 속력이 입력값과 같게 나오는 것으로 확인했다.

11

발표 대본

2장 · 약 60초

방금 그 식이 코드로는 열 줄 남짓입니다.

첫 번째 호출이 동체에서 NED 로 돌리는 것입니다. 속도를 (V, 0, 0) 으로 넣습니다. 앞으로만 가고 옆이나 위로는 안 간다는 뜻입니다.

두 번째 호출이 NED 에서 ECEF 입니다.

오른쪽 카드 세 개가 이 열 줄에서 실제로 걸렸던 부분입니다.

첫째, 두 번째 함수 인자에 0이 세 개 들어가 있습니다. 이 함수는 회전만 해 주는 게 아니라 기준점만큼 평행이동까지 해 줍니다. 위치를 옮길 때는 그게 맞는데, 속도는 위치가 아니라 방향이라서 그 이동분을 빼야 합니다. 그래서 기준점을 원점으로 넣었습니다. 처음에 이걸 모르고 위치처럼 호출했다가, 표적이 지구 반대편으로 날아가는 걸 보고 알았습니다.

둘째, 첫 번째 함수의 인자 순서가 Roll, Yaw, Pitch 입니다. 자세각 구조체의 필드 순서와 달라서 한 번 틀렸던 자리라 주석으로 남겼습니다.

셋째가 방금 말씀드린 속력 확인입니다.

02. 시간에 따라 움직인다

공식 ② — 한 스텝 전진시킨다

어느 지점의 속도를 쓰느냐로 결과가 갈립니다.

한 스텝 전진 — 어느 지점의 속도를 쓸 것인가

출발점 속도로 그냥 간다



반 스텝 가서 속도를 다시 재고 간다



회색 점이 반 스텝 지점이다. 그 자리에서 켄 속도가 구간 전체의 평균에 훨씬 가깝기 때문에, 같은 간격으로도 덜 벗어난다.

쓰는 식

$$\mathbf{v}_{mid} = \mathbf{v}(\mathbf{p}_k + \frac{\Delta t}{2} \mathbf{v}_k, \quad \Theta_k + \frac{\Delta t}{2} \omega)$$

$$\mathbf{p}_{k+1}^c = \mathbf{p}_k^c + \mathbf{v}_{mid} \Delta t$$

반 스텝 지점의 위치와 자세로 속도를 한 번 더 만들어 쓴다.

왜 굳이 두 번 계산하는가

표적은 늘 돌고 있고, 지구도 둥글다.
한 스텝 안에서도 방향이 계속 바뀐다.

같은 0.1 초 간격에서 60 초 뒤 위치 오차

출발점 속도로

16.6 m

중점 속도로

→ 1.36 cm

약 1,200 배

12

발표 대본

2장 · 약 65초

4단계에서 쓰는 공식입니다. 위치에 속도 곱하기 0.1초를 더한다고 말씀드렸는데, 여기에 한 가지 선택이 남아 있습니다. 어느 지점의 속도를 쓸 것인가입니다.

왼쪽 위가 제일 단순한 방법입니다. 출발할 때의 속도로 0.1초를 그냥 갑니다. 그런데 표적이 도는 중이면 0.1초 사이에도 방향이 계속 바뀝니다. 출발할 때 속도를 끝까지 쓰면 회색 파선, 그러니까 실제 궤적에서 점점 벌어집니다.

왼쪽 아래가 실제로 쓴 방법입니다. 일단 반 스텝만 살짝 가 봅니다. 회색 점입니다. 거기서 속도를 다시 재고, 그 속도로 처음 위치에서 한 스텝을 갑니다. 반 스텝 지점의 속도가 구간 전체의 평균에 훨씬 가깝기 때문에, 같은 0.1초를 써도 훨씬 덜 벗어납니다. 그림을 보시면 초록 선이 회색 파선에 거의 붙어 있습니다.

오른쪽 아래가 실제 측정값입니다. 2G 로 20초 도는 시나리오를 같은 0.1초 간격으로 60초 돌렸을 때, 출발점 속도만 쓰면 16.6미터 벗어나고, 중점 속도를 쓰면 1.36센티미터입니다. 천 배가 넘습니다. 계산은 두 배 드는데 오차는 천분의 일이 되니까 충분히 남는 장사입니다.

02. 시간에 따라 움직인다

코드 ② — 한 스텝 전진시킨다

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발췌)

```

halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세 — 각속도가 구간 내 상수라 바로 더한다
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...

```

자세만 반 스텝 돌리면 안 되는 이유

"북쪽"이라는 기준은 표적이 서 있는 자리에서 정해진다. 자세만 돌리고 위치를 두면 방향의 기준이 뒤에 남아, 곧게 날려도 고도가 조금씩 뜬다. 위치도 함께 반 스텝 옮긴 이유다.

계산이 깨져도 상태는 지킨다

사본에서 전부 계산하고 값이 정상인지 확인한 뒤, 모두 성공했을 때만 한 번에 반영한다.

알고도 남겨 둔 한계 — 짐벌락

Pitch 가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐서 Yaw 기동이 궤적을 바꾸지 못한다.

13

발표 대본

2장 · 약 65초

코드로는 이렇습니다. 주석에 붙인 번호 순서대로 보시면 됩니다.

①에서 반 스텝 뒤의 자세를 구합니다. 각속도가 기동 구간 안에서는 상수라 그냥 더하면 됩니다.

②에서 반 스텝 뒤의 위치를 구합니다. 출발할 때 속도로 잠깐 예측하는 겁니다.

③이 핵심입니다. 그 자리, 그 자세에서 속도를 다시 만듭니다. 이게 이 스텝의 대표 속도입니다.

④에서 그 속도로 한 스텝을 옮깁니다.

오른쪽 첫 번째 카드가 ②를 처음에 빼먹었던 이야기입니다. 자세만 반 스텝 돌리고 위치는 그대로 두고 속도를 만 들었더니, 곧게 날아가는 표적의 고도가 조금씩 올라갔습니다. 원인은 "북쪽"이라는 기준이 표적이 서 있는 자리에 서 정해진다는 데 있었습니다. 표적이 앞으로 나아갔으면 방향의 기준도 같이 따라가야 하는데, 자세만 돌리고 위치를 두니까 기준이 뒤에 남은 겁니다. 그래서 위치도 같이 반 스텝 옮기게 고쳤습니다.

두 번째 카드. 계산은 전부 사본에서 하고, 값이 정상인지 확인한 뒤에 모두 성공했을 때만 한 번에 반영합니다. 중간에 하나가 깨져도 상태가 반쯤 갱신된 채로 남지 않습니다.

세 번째는 알고도 남겨 둔 한계입니다. Pitch 가 90도 근처로 가면 Yaw 축이 속도 방향과 겹쳐서 Yaw 기동이 궤적을 못 바꿉니다. 짐벌락이라고 하는 현상인데, 오일러각을 직접 적분하는 방식의 구조적 한계입니다. 이번 과제 조건에서는 Pitch 가 그 근처까지 가지 않아서 그대로 뒀습니다.

03. 결과를 확인한다

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60s · 간격 0.1s

스텝 600 회, 표본 601 개.

시각은 이전 값에 더해 가지 않고
스텝 번호 × 간격으로 매겨
오차가 쌓이지 않게 했다.

답을 미리 알 수 있는 조건이다

기동이 하나도 없다. 그래서 손으로
답을 낼 수 있다.

대함은 정서진 → 위도 그대로,
대공은 정남진 → 경도 그대로.

플랫폼 속력이 0 인 점

각속도 식은 속력으로 나눈다.

그 나눗셈은 기동이 걸린 구간
안에서만 하는데, 플랫폼은 기동이
없어 그 자리를 지나가지 않는다.

14

발표 대본

3장 · 약 45초

과제 문서 2항의 값을 그대로 넣었습니다.

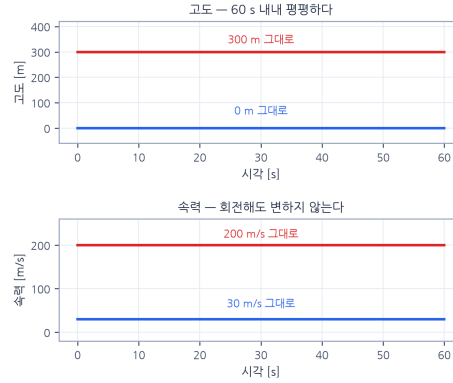
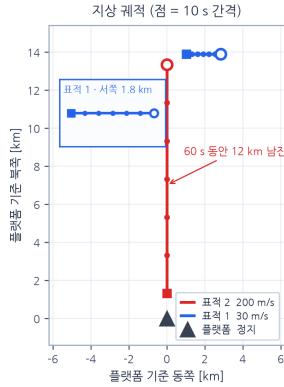
플랫폼은 32도, 126도에 정지해 있습니다. 대함 표적은 그보다 북동쪽에서 서쪽으로 30 m/s, 대공 표적은 북쪽 300미터 고도에서 남쪽으로 200 m/s 입니다. 60초를 0.1초 간격으로 돌리니까 스텝은 600번, 표본은 처음 것을 포함해 601개입니다.

가운데 카드를 봐 주십시오. 이 조건에는 기동이 하나도 없습니다. 처음에는 미묘하다고 생각했는데, 검증 입장에서 는 오히려 좋은 조건입니다. 답을 손으로 낼 수 있기 때문입니다. 대함은 정서쪽으로만 가니까 위도가 안 변해야 하고, 대공은 정남쪽으로만 가니까 경도가 안 변해야 합니다. 둘 다 고도는 그대로여야 합니다.

시각은 이전 시각에 0.1을 계속 더하는 대신 스텝 번호 곱하기 간격으로 매겼습니다. 더하기를 600번 반복하면 미세한 오차가 쌓이는데, 곱하기 한 번이면 그럴 일이 없습니다.

03. 결과를 확인한다

결과 ① — 명세 조건 60 초



이동 거리

대함 1800 m · 대공 12000 m —
속력 × 시간과 같다.

방향

대함은 위도가, 대공은 경도가
60 초 내내 그대로였다.

고도 · 속력

널은 값이 끝까지 유지됐다 —
회전이 크기를 바꾸지 않는다는 뜻.

15

발표 대본

3장 · 약 60초

결과입니다. 왼쪽이 플랫폼을 원점으로 잡은 동-북 평면이고, 점 하나가 10초입니다.

빨간 게 대공 표적입니다. 북쪽 13.3킬로미터 지점에서 출발해서 똑바로 남쪽으로 내려왔습니다. 60초 동안 12킬로미터입니다. 200 곱하기 60이 12000이니깐 맞습니다.

파란 게 대함 표적인데, 이 축척에서는 거의 점입니다. 왼쪽 위에 확대해 봤습니다. 서쪽으로 1.8킬로미터 갔고, 30 곱하기 60이 1800이니 이것도 맞습니다.

오른쪽 위가 고도입니다. 60초 내내 평평합니다. 대공은 300미터, 대함은 0미터 그대로입니다.

오른쪽 아래가 속력입니다. 널은 값이 끝까지 유지됐습니다. 회전행렬을 여러 번 거쳐도 크기가 안 변한다는 걸 결과로 확인한 셈입니다.

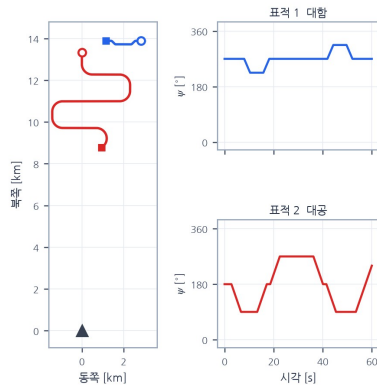
아래 카드 세 개가 그 세 가지를 정리한 겁니다. 이동 거리, 방향, 고도와 속력. 전부 손으로 계산한 값과 맞았습니다.

03. 결과를 확인한다

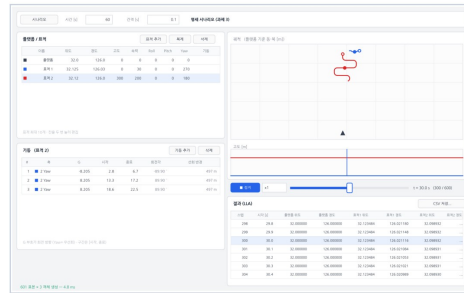
결과 ② — 기동 시연과 운용 화면

초기값 · 시간 · 간격은 명세 그대로 두고, 기동표 10 줄만 추가했습니다.

기동 10줄을 얹은 60 s 궤적



TargetSim — 값을 바꿔 가며 확인하는 화면



고치면 곧바로 다시 돈다
표의 칸을 고치면 0.25 s 뒤 자동으로 다시 계산한다. 601 표본 × 3 객체가 4.8 ms 라 기다릴 일이 없다.

그림에서 끌어서 배치
시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 반영된다. 회전각과 선회 반경은 G 로부터 계산.

결과는 표 · CSV · 그림
시각별 LLA 를 표로 보고 슬라이더로 되감고 CSV 로 내보낸다. 오늘 숫자가 전부 여기서 나온 CSV 다.

16

발표 대본

3장 · 약 55초

기동이 없는 조건만 보면 직선 두 개라 심심해서, 같은 초기값에 기동표 열 줄만 얹어서 한 번 더 돌려 봤습니다.

왼쪽이 그 결과입니다. 대공 표적이 지그재그로 내려옵니다. 오른쪽 방위각 그래프에서 계단처럼 보이는 구간이 기동이 걸린 구간이고, 나머지는 직진입니다. 표에 적은 대로 정확히 그 시각에 돌았습니다.

가운데가 실제 운용 화면입니다. 위쪽 표에서 표적을 추가하고 값을 고칠 수 있고, 아래 표에서 기동을 넣습니다. 오른쪽에 궤적과 고도가 그려지고, 슬라이더로 시각을 되감을 수 있습니다.

오른쪽 첫 번째 메모를 하나 짚겠습니다. 표의 칸을 고치면 0.25초 뒤에 자동으로 다시 계산합니다. 601표본에 표적 3개를 계산하는 데 4.8밀리초라서, 값을 바꾸고 기다릴 일이 없습니다. 값을 이리저리 바꿔 보면서 감을 잡을 수 있게 하려고 이렇게 만들었습니다.

결과는 CSV 로 내보낼 수 있고, 오늘 보여 드린 숫자가 전부 그 CSV 에서 나온 것입니다.

03. 결과를 확인한다

정리

요구사항과 구현을 한 장에 놓고 보겠습니다.

과제 요구사항	구현	확인
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_heading	ST_TargetInit — 방향은 자세각이 담당	통과
기동 = G · 축 · 시작 · 종료	ST_TargetManeuver 4 필드, 구간 겹침 검사	통과
ECEF 에서 상태 갱신	위치 · 속도는 ECEF, 출력용 LLA 병행 보관	통과
V_heading → Vx, Vy, Vz (추가 고민)	동체 → NED → ECEF, 회전 두 번	속력 보존
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본	무수정
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 · 고도 · 방위 그림	601 표본

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 — 사원수로 바꾸면 해소된다
- 힘이 아니라 속도를 직접 주는 모델 — 탄도 표적에는 부적합
- 속력이 일정해 가감속 기동은 아직 표현하지 못한다

이 위에 이어 붙일 것

- 궤적에 측정 잡음을 없으면 그대로 추적 필터 입력이 된다
- 안테나 기준 거리 · 방위 · 고각으로 바꾸면 탐지 모의가 된다
- 참값을 알고 있으니 추적 오차를 바로 숫자로 낼 수 있다

17

발표 대본

마무리 · 약 55초

요구사항과 구현을 한 장에 놓고 보겠습니다. 표적 10개와 기동 30개 상한, 위치와 속도 표현, 기동 네 필드, ECEF 갱신, 그리고 추가 고민 항목이었던 속도 변환까지 전부 반영했습니다. 좌표변환 코드는 한 줄도 고치지 않았습니다.

왼쪽 아래가 남겨 둔 한계입니다. 짐벌락은 사원수로 바꾸면 없어지고, 힘을 풀지 않는 운동학 모델이라 탄도 표적에는 맞지 않습니다. 속력이 일정하게 유지되는 구조라 가감속 기동도 아직 표현하지 못합니다. 셋 다 이번 과제 범위에서는 문제가 안 됐지만, 알고 있다는 것은 말씀드리는 게 맞다고 봤습니다.

오른쪽이 이 위에 이어 붙일 수 있는 것들입니다. 지금 나오는 건 깨끗한 참값이니까, 여기에 측정 잡음을 없으면 그대로 추적 필터 입력이 됩니다. 안테나 기준 거리와 방위, 고각으로 바꾸면 탐지 모의가 되고요. 무엇보다 참값을 우리가 알고 있으니까, 추적 결과가 얼마나 벗어났는지를 바로 숫자로 낼 수 있습니다. 처음에 말씀드린 "참값을 만들어 뒤쪽에 넘긴다" 가 이걸 말한 것입니다.

**발표 대본**

마무리 · 약 15초

이상입니다. 감사합니다.

질문 있으시면 답변드리겠습니다.